

2026- 1학기 블록체인실습
프로젝트 제안서

Workout Reward Token DApp

운동 기록 인증 · AI 사진 검증 · 블록체인 보상 토큰 지급 DApp



과 목 명: 블록체인실습
담 당 교 수: 김형중 교수님
학 번: 20222117
학 과: 정보보안암호수학과
이 름: 안두홍

목 차

- I. 프로젝트 주제
 - 1. Workout Reward Token DApp의 개요
 - 2. 프로젝트의 기본 목표
- II. 주제 선정 이유
 - 1. 운동 인증 주제를 선택한 이유
 - 2. 블록체인 보상 구조와의 연결성
 - 3. 기존 수업 실습과의 차별점
- III. 사용 기술 및 구현 환경
 - 1. Solidity와 ERC- 20 토큰
 - 2. MetaMask와 테스트넷
 - 3. ethers.js와 프론트엔드
- IV. 프로젝트 핵심 기능
 - 1. 지갑 연결 기능
 - 2. 운동 기록 입력 기능
 - 3. Workout Token 보상 기능
 - 4. 운동 기록 조회 기능
- V. 구현 절차와 시연 계획
 - 1. 스마트 컨트랙트 작성 및 배포
 - 2. 프론트엔드 제작과 지갑 연결
 - 3. 운동 인증 및 토큰 지급 시연
- VI. 예상 결과물과 확장 방향
 - 1. 예상 결과물
 - 2. 현재 한계점
 - 3. 향후 확장 가능성
- VII. 결론
- VIII. 참고자료

I. 프로젝트 주제

1. Workout Reward Token DApp의 개요

이번 프로젝트의 주제는 Workout Reward Token DApp이다. 이 프로젝트는 사용자가 자신의 운동 기록을 입력하면, 운동 시간과 강도에 따라 ERC- 20 기반 보상 토큰을 지급받는 블록체인 DApp이다.

사용자는 웹 화면에서 MetaMask 지갑을 연결한 뒤, 운동 종류, 운동 시간, 운동 강도 등을 입력한다. 입력된 운동 기록이 기준을 만족하면 스마트 컨트랙트가 사용자에게 Workout Token(WKT)을 지급한다. 사용자는 자신의 지갑 주소, 보유 토큰 수량, 운동 인증 기록을 웹 화면에서 확인할 수 있다.

이 프로젝트는 단순히 토큰을 발행하거나 전송하는 실습에서 끝나는 것이 아니라, 운동이라는 실제 생활 습관을 블록체인 보상 구조와 연결해 보는 데 목적이 있다.

2. 프로젝트의 기본 목표

이 프로젝트의 기본 목표는 수업에서 배운 스마트 컨트랙트, ERC-20 토큰, MetaMask, 테스트넷, ethers.js를 활용하여 하나의 작동 가능한 DApp 흐름을 직접 구현하는 것이다.

구체적인 목표는 다음과 같다. 첫째, Solidity를 이용해 운동 기록을 저장하고 보상 토큰을 지급하는 스마트 컨트랙트를 작성한다. 둘째, ERC-20 기반의 Workout Reward Token(WKT)을 만든다. 셋째, HTML, CSS, JavaScript를 이용해 사용자가 조작할 수 있는 간단한 웹 화면을 제작한다. 넷째, ethers.js를 이용해 프론트엔드와 스마트 컨트랙트를 연결한다. 다섯째, MetaMask를 통해 트랜잭션을 실행하고, 운동 인증 후 토큰이 지급되는 과정을 시연한다.

II. 주제 선정 이유

1. 운동 인증 주제를 선택한 이유

운동은 꾸준히 하는 것이 가장 중요하지만, 실제로는 지속하기 어렵다. 많은 사람들이 운동 계획을 세우지만 시간이 지나면 기록을 남기지 않거나 동기부여가 떨어지는 경우가 많다. 그래서 운동 기록에 보상 구조를 붙이면 사용자가 더 꾸준히 운동할 수 있는 동기를 얻을 수 있다고 생각했다.

일반적인 운동 기록 앱은 사용자가 운동 내용을 입력하고 확인하는 데서 끝난다. 하지만 이 프로젝트는 운동 기록을 입력했을 때 토큰이라는 보상이 지급되도록 하여, 운동 기록이 단순한 메모가 아니라 블록체인 위의 보상 활동으로 이어지게 만든다.

2. 블록체인 보상 구조와의 연결성

블록체인은 거래 기록을 투명하게 남길 수 있고, 스마트 컨트랙트를 통해 정해진 조건에 따라 자동으로 동작할 수 있다. 이 특징을 운동 인증에 연결하면, 사용자가 운동 기록을 남겼을 때 정해진 기준에 따라 자동으로 보상이 지급되는 구조를 만들 수 있다.

예를 들어 10분 이상 운동하면 5 WKT, 30분 이상 운동하면 10 WKT, 60분 이상 운동하면 20 WKT를 지급하도록 설정할 수 있다. 또한 고강도 운동을 선택하면 추가 보상을 지급하는 방식으로 운동 조건을 세분화할 수 있다. 이처럼 블록체인 보상 구조를 활용하면 사용자의 행동과 토큰 지급을 하나의 흐름으로 연결할 수 있다.

3. 기존 수업 실습과의 차별점

수업에서는 MetaMask, 테스트넷, 스마트 컨트랙트, Hardhat, ERC- 20 토큰, ethers.js 등을 다루었다. 기존 실습이 토큰 발행, 전송, 컨트랙트 배포의 기본 구조를 배우는 데 초점이 있었다면, 이번 프로젝트는 그 내용을 실제 서비스 흐름으로 확장한다는 점에서 차이가 있다.

단순히 ERC- 20 토큰을 만드는 것이 아니라, 사용자의 운동 기록을 입력받고, 조건을 판단한 뒤, 보상 토큰을 지급하는 DApp 형태로 구현한다. 즉, 수업에서 배운 내용을 그대로 반복하는 것이 아니라 운동 인증, AI 사진 검증, 조건 확인, 토큰 지급, 기록 조회라는 새로운 흐름으로 확장하는 프로젝트. 특히 외부 AI API(Claude Vision)를 블록체인 보상 흐름과 연결한다는 점이 핵심 차별점이다.

III. 사용 기술 및 구현 환경

1. Solidity와 ERC- 20 토큰

스마트 컨트랙트는 Solidity로 작성한다. Solidity는 이더리움 계열 블록체인에서 스마트 컨트랙트를 작성할 때 사용하는 언어이다. 이번 프로젝트에서는 Solidity를 이용해 운동 기록을 저장하고, 조건에 따라 ERC- 20 토큰을 지급하는 컨트랙트를 만든다.

토큰은 ERC- 20 표준을 사용한다. ERC- 20은 이더리움에서 많이 사용되는 토큰 표준이며, balanceOf, transfer, approve, allowance 같은 기본 기능을 제공한다. 이번 프로젝트에서는 OpenZeppelin의 ERC- 20 라이브러리를 활용하여 기본 토큰 기능을 안정적으로 구현할 계획이다.

토큰 이름은 Workout Reward Token, 토큰 심볼은 WKT로 설정한다. 이 토큰은 실제 금전적 목적이 아니라 운동 인증에 대한 보상 포인트로 사용된다.

2. MetaMask와 테스트넷

사용자는 MetaMask 지갑을 통해 DApp에 접속한다. MetaMask는 사용자의 지갑 주소를 웹 페이지와 연결하고, 스마트 컨트랙트 함수 실행 시 트랜잭션을 승인하는 역할을 한다.

배포 네트워크는 Sepolia 또는 GIIWA Sepolia 테스트넷을 사용할 예정이다. 테스트넷을 사용하는 이유는 실제 자산을 사용하지 않고도 스마트 컨트랙트 배포와 트랜잭션 실행을 연습할 수 있기 때문이다.

프로젝트 시연에서는 테스트넷에 컨트랙트를 배포하고, MetaMask로 운동 인증 트랜잭션을 실행하는 과정을 보여줄 예정이다.

3. ethers.js와 프론트엔드

프론트엔드는 HTML, CSS, JavaScript로 제작한다. JavaScript는 웹 브라우저에서 동작하는 언어이며, 버튼 클릭, 입력값 처리, 화면 갱신 같은 기능을 담당한다.

블록체인 연결에는 ethers.js를 사용한다. ethers.js는 JavaScript 앱이 이더리움 블록체인과 통신할 수 있도록 도와주는 라이브러리이다. 프론트엔드에서는 ethers.js를 이용해 MetaMask와 연결하고, 배포된 스마트 컨트랙트의 함수를 호출한다.

프론트엔드에서 구현할 기능은 지갑 연결, 운동 정보 입력, recordWorkout 함수 호출, WKT 잔액 조회, 운동 기록 조회이다.

4. Claude Vision API

Anthropic의 Claude API(claude-sonnet-4-20250514 모델)를 활용하여 운동 사진 인증 기능을 구현한다. 사용자가 운동 사진을 업로드하면 프론트엔드가 이미지를 base64로 인코딩하여 API에 전달한다. Claude는 해당 이미지가 운동 관련 사진인지 판단하고, { isWorkout: true/false, reason: "..." } 형식의 JSON으로 결과를 반환한다. 프론트엔드는 이 결과를 받아 추가 보상 여부를 결정한 뒤 스마트 컨트랙트를 호출한다.

IV. 프로젝트 핵심 기능

1. 지갑 연결 기능

첫 번째 핵심 기능은 MetaMask 지갑 연결이다. 사용자가 웹 페이지에 접속하면 Connect Wallet 버튼을 눌러 자신의 지갑을 연결한다. 지갑 연결 후에는 화면에 사용자의 지갑 주소가 표시된다. 또한 현재 사용자가 보유한 WKT 토큰 잔액을 조회하여 보여준다.

예상 화면에는 지갑 주소와 보유 WKT 수량이 표시된다. 이 기능을 통해 사용자는 자신이 어떤 지갑으로 운동 인증을 진행하고 있는지 확인할 수 있다.

2. 운동 기록 입력 기능

두 번째 기능은 운동 기록 입력이다. 사용자는 웹 화면에서 자신의 운동 정보를 입력한다. 입력 항목은 운동 종류, 운동 시간, 운동 강도, 운동 메모로 구성한다.

운동 종류에는 헬스, 러닝, 푸쉬업, 스쿼트 등을 선택할 수 있도록 한다. 운동 시간은 분 단위로 입력하고, 운동 강도는 easy, normal, hard 중 하나를 선택하도록 한다. 운동 시간은 보상 토큰 수량을 결정하는 핵심 기준으로 사용되고, 운동 강도는 추가 보상을 계산하는 기준으로 사용한다.

3. Workout Token 보상 기능

세 번째 기능은 운동 조건에 따른 보상 토큰 지급이다. 사용자가 운동 인증 버튼을 누르면 스마트 컨트랙트의 recordWorkout 함수가 실행된다. 이 함수는 운동 시간을 확인하고, 보상 수량을 계산한 뒤, 사용자 지갑에 WKT 토큰을 지급한다.

보상 기준은 10분 이상 운동하면 5 WKT, 30분 이상 운동하면 10 WKT, 60분 이상 운동하면 20 WKT로 설정한다. 고강도 운동을 선택하면 추가로 5 WKT를 지급한다. 하루에 여

러 번 반복해서 보상을 받는 문제를 막기 위해 lastClaimTime을 저장하고, 하루에 한 번만 보상을 받을 수 있도록 제한한다. 또한 운동 사진을 업로드하여 Claude AI 인증을 통과하면 추가로 3 WKT를 지급한다.

4. 운동 기록 조회 기능

네 번째 기능은 운동 기록 조회이다. 사용자가 운동 인증을 완료하면 해당 기록은 스마트 컨트랙트에 저장된다. 저장되는 정보는 운동 인증을 한 지갑 주소, 운동 종류, 운동 시간, 운동 강도, 지급된 토큰 수량, 인증 시간이다.

운동 기록처럼 계속 남겨야 하는 데이터는 구조체와 배열을 이용해 저장한다. 사용자는 웹 화면에서 자신의 최근 운동 기록과 지급받은 토큰 수량을 확인할 수 있다.

V. 구현 절차와 시연 계획

1. 스마트 컨트랙트 작성 및 배포

먼저 Solidity로 WorkoutRewardToken.sol 컨트랙트를 작성한다. 컨트랙트에는 ERC-20 토큰 생성, 운동 기록 구조체 작성, 운동 기록 배열 저장, 사용자별 운동 기록 관리, 운동 시간과 강도에 따른 보상 계산, 하루 1회 보상 제한, WKT 토큰 지급 기능을 포함한다.

처음에는 Remix에서 컴파일과 배포를 진행한다. Remix는 웹 기반으로 빠르게 스마트 컨트랙트를 작성하고 테스트할 수 있기 때문에 초기 실습에 적합하다. 이후 시간이 가능하면 Hardhat 프로젝트로 확장하여 배포 스크립트와 프론트엔드 연결까지 정리한다.

2. 프론트엔드 제작과 지갑 연결

다음으로 HTML, CSS, JavaScript를 이용해 간단한 웹 화면을 제작한다. 웹 화면에는 지갑 연결 버튼, 현재 지갑 주소 표시, WKT 토큰 잔액 표시, 운동 종류 입력, 운동 시간 입력, 운동 강도 선택, 운동 인증 버튼, 최근 운동 기록 표시 기능을 넣는다.

프론트엔드에서는 ethers.js를 이용해 MetaMask와 스마트 컨트랙트를 연결한다. 사용자가 운동 인증 버튼을 누르면 프론트엔드가 스마트 컨트랙트의 recordWorkout 함수를 호출하고, MetaMask에서 트랜잭션 승인을 요청한다.

3. 운동 인증 및 토큰 지급 시연

시연에서는 완성도보다 실제 작동 흐름을 보여주는 데 집중한다. 시연 순서는 웹 페이지 실행, MetaMask 지갑 연결, 운동 종류와 운동 시간 및 운동 강도 입력, 운동 사진 업로드 → Claude AI 분석 결과 확인, 운동 인증 버튼 클릭, MetaMask 트랜잭션 승인, WKT 토큰 잔액 증가 확인, 운동 기록 조회, 테스트넷 Explorer에서 트랜잭션 확인 순서로 진행한다.

예를 들어 운동 종류를 헬스, 운동 시간을 60분, 운동 강도를 hard로 입력하면 기본 보상 20 WKT와 고강도 추가 보상 5 WKT를 합쳐 사진 인증 통과 시 추가 3 WKT를 포함해 총

28 WKT가 지급된다. 이 과정을 통해 운동 기록이 단순한 입력값으로 끝나는 것이 아니라, 스마트 컨트랙트 실행과 토큰 지급으로 이어진다는 점을 보여줄 수 있다.

VI. 예상 결과물과 확장 방향

1. 예상 결과물

이번 프로젝트의 예상 결과물은 운동 인증 기능이 있는 웹 DApp이다. 사용자는 웹 화면에서 지갑을 연결하고 운동 기록을 입력할 수 있다. 또한 ERC-20 기반의 Workout Reward Token(WKT) 스마트 컨트랙트가 만들어지며, 이 컨트랙트는 운동 기록을 저장하고 운동 시간과 강도에 따라 토큰을 지급한다.

GitHub 저장소에는 스마트 컨트랙트 코드, 프론트엔드 코드, README, 실행 방법, 시연 캡처를 정리한다. 시연에서는 지갑 연결, 운동 인증, 트랜잭션 승인, 토큰 잔액 증가, 운동 기록 조회를 보여준다.

2. 현재 한계점

현재 버전의 가장 큰 한계는 사용자가 운동 시간을 직접 입력한다는 점이다. 사용자가 실제로 운동하지 않았더라도 임의로 시간을 입력하면 보상을 받을 수 있다. 따라서 완전한 인증 시스템이라고 보기는 어렵다.

이번 프로젝트에서는 Claude Vision API를 활용한 사진 인증으로 이를 보완하였다. 다만 API 키가 프론트엔드에 노출되는 구조이므로, 실제 서비스라면 백엔드 프록시 서버를 두어야 한다는 한계가 있다.

3. 향후 확장 가능성

운동 사진 인증은 이번 프로젝트에서 Claude Vision API를 통해 1차 구현하였으며, 추후 백엔드 프록시 서버와 오라클 연동을 통해 보안성을 높일 수 있다.

또한 GPS 기반 러닝 인증 기능을 추가할 수 있다. 사용자의 이동 거리와 시간을 기반으로 러닝 기록을 확인하고, 일정 거리 이상 달렸을 때 보상을 지급할 수 있다.

헬스장 QR 체크인 기능도 확장 가능하다. 헬스장에 QR 코드를 두고, 사용자가 QR을 스캔하면 출석 인증이 되도록 만들 수 있다.

이 외에도 누적 운동 횟수에 따라 Bronze, Silver, Gold 같은 NFT 배지를 발급하거나, 사용자별 누적 WKT 보유량과 운동 인증 횟수를 기준으로 랭킹 시스템을 만들 수 있다. 이러한 기능을 추가하면 단순한 운동 기록 DApp에서 운동 커뮤니티 또는 습관 형성 서비스로 확장할 수 있다.

VII. 결론

Workout Reward Token DApp은 수업에서 배운 블록체인 기술을 운동 인증이라는 생활

속 주제와 연결해 보는 프로젝트이다. 이 프로젝트는 단순히 ERC- 20 토큰을 발행하거나 전송하는 것에서 끝나지 않고, 사용자의 행동을 기록하고 그에 따라 보상을 지급하는 서비스 흐름을 구현한다는 점에서 의미가 있다.

이번 프로젝트를 통해 Solidity 스마트 컨트랙트 작성, ERC- 20 토큰 발행, MetaMask 지갑 연결, ethers.js를 이용한 컨트랙트 호출, 테스트넷 배포와 트랜잭션 확인 과정을 종합적으로 경험할 수 있다.

현재는 사용자가 운동 시간을 직접 입력하기 때문에 완전한 인증이 어렵다는 한계가 있으며, Claude Vision API 사진 인증으로 이를 부분적으로 보완하였다. 이후 GPS 인증, QR 체크인, NFT 배지 기능을 추가하면 더 실제적인 서비스로 발전시킬 수 있다.

따라서 이 프로젝트는 수업에서 배운 내용을 바탕으로 직접 구현 가능한 범위 안에서 블록체인 기술을 실생활 문제와 연결해 보는 팀 프로젝트라고 할 수 있다.

VIII. 참고 자료

1. 김형중, 「Blockchain Lab #1」, 국민대학교 암호화폐와 디지털 금융 수업자료, 2026.
2. 김형중, 「Blockchain Lab #2」, 국민대학교 암호화폐와 디지털 금융 수업자료, 2026.
3. 김형중, 「Blockchain Lab #4」, 국민대학교 암호화폐와 디지털 금융 수업자료, 2026.
4. 김형중, 「Blockchain Lab #5」, 국민대학교 암호화폐와 디지털 금융 수업자료, 2026.
5. OpenZeppelin Docs, ERC-20 Documentation.
6. ethers.js Documentation.
7. MetaMask Documentation.
8. Anthropic, Claude API Documentation. <https://docs.anthropic.com>
9. Anthropic, Vision Documentation. <https://docs.anthropic.com/en/docs/build-with-claude/vision>